

Rekolt Application

Application Scope and Assumptions

The Rekolt app is a CLI application written in Java to help the treasurer at REKOLT Planters' Cooperative pay the member farmers correctly what they are owed while also keeping a season report recorded in case any complaints arise. The application records each delivery brought to the collection point, applies the cooperative's five payment rules identically to every load, answers the committee's end-of-season questions on screen, and produces one Microsoft Word season report covering the whole membership.

What the system covers

- ❖ Capturing one delivery at a time: member identifier, member name, produce code, mass, quality score and week, with every field validated at the prompt.
- ❖ Grading the load from its quality score, and computing base value, graded value, category value, commission, transport levy and net payable.
- ❖ Holding the season in memory: all deliveries, the deliveries and total payable per member, the distinct members seen, and the weekly volume grid.
- ❖ Answering on screen: total payment per member, the weekly volume grid in kilograms, and the highest-value deliveries.
- ❖ Writing one output/season-report.docx with a section per member and a closing season total.

What the system deliberately does not cover

- ❖ No persistence of delivery data. Deliveries live in the Java collections built in objective 3 for as long as the process runs and are lost when it exits. Nothing is read from a file at startup and nothing is written back.
- ❖ No user accounts, no login, no roles. One person operates the program and it behaves the same for all of them.
- ❖ No editing or deletion of a recorded delivery. Correcting or removing a false document is not a requirement for this project.
- ❖ No per-member file. Every member's data is recorded and kept in one general season report
- ❖ No graphical interface was implemented, or database, and no currency other than the Mauritian rupee was used.

Assumptions Made while Building The Application

- ❖ Delivery data is not persisted between runs. Only the Word report and the run log touch the file system.
- ❖ The four produce codes, their base prices and their categories are fixed for the season and are held in the program, not entered by the user.
- ❖ A member identifier uniquely identifies a planter. If the same identifier is entered with two different spellings of the name, the first name recorded for that identifier is the one kept.
- ❖ The season is at most twenty weeks and every delivery falls in exactly one of them.
- ❖ The quality score submitted as an input is final. The program does not re-score a load or question a score inside the valid range.
- ❖ All money is in Mauritian rupees. There is no conversion needed, and no tax other than the cooperative's commission and levy is applied.
- ❖ The cooperative has at most 50 members and maybe 100 deliveries per season at most
- ❖ The program runs on JDK 21 or later.

Program Requirements

Each requirement below is written so that it can be checked by running the program and observing if it fulfills the requirement.

Functional requirements

Menu Session

Ref	Requirement	How it is checked
FR1	The program shall display a menu of four options: record a delivery, season figures on screen, generate the season report, and exit. It shall redisplay it after every completed option until exit is chosen.	Options 1, 2 and 3 return to the menu; option 4 prints a closing message and ends.
FR2	The menu shall accept only a whole number from 1 to 4.	Any other number, a String, or an empty line are each treated as invalid and the user is re-prompted.
FR3	Recording a delivery shall prompt for six values in this order: member identifier, member name, produce code, mass, quality score, week	The prompts appear in that order in the transcript.

Input validation

Ref	Requirement	How it is checked
FR4	A member identifier shall be accepted only if it is the letter M, a hyphen, then exactly four digits.	M-0042 accepted; M-42, M-00423, m-0042, empty rejected
FR5	A member name shall be accepted only if it contains at least one non-whitespace character.	An empty line and a line of spaces are both rejected
FR6	A produce code shall be accepted only if it is MZE, BNS, POT or TEA in either case, and shall be stored in upper case.	bns accepted and stored as BNS while RIC is rejected
FR7	A mass shall be accepted only if it is a number strictly greater than 0 and not more than 5000 kilograms.	0.01 and 5000 are accepted while 0, -12, 5000.01 and abc are refused.
FR8	A quality score shall be accepted only if it is a whole number from 0 to 100 inclusive.	0 and 100 are accepted while -1, 101 and 72.5 are rejected
FR9	A week shall be accepted only if it is a whole number from 1 to 20 inclusive.	1 and 20 are accepted while 0 and 21 rejected
FR10	Any refused entry shall print one short line giving the acceptable range and shall re-prompt the same field. No entry the user can type shall end the program with a stack trace.	A transcript of one deliberately wrong entry per field and advice on how to correct it. All six prompts are recovered like that.

Grading and payment

Ref	Requirement	How it is checked
FR11	A quality score of 85 to 100 shall grade A, 70 to 84 shall grade B, 50 to 69 shall grade C, and 49 or below shall grade REJECT.	Scores 100, 85, 84, 70, 69, 50, 49 and 0 give A, A, B, B, C, C, REJECT, REJECT.
FR12	The grade multipliers shall be A $\times 1.15$, B $\times 1.00$, C $\times 0.85$ and REJECT $\times 0.00$.	Each grade's multiplier printed on step 2 of the netpayable breakdown
FR13	Base value shall be mass in kilograms multiplied by the base price of the produce: MZE 30.00, BNS 90.00, POT 45.00, TEA 25.00 MUR per kilogram.	236 kg of BNS gives 21,240.00 MUR
FR14	Graded value shall be the base value multiplied by the grade multiplier.	$21,240.00 \times 1.15 = 24,426.00$. (1.15 was used because an A grade was given to the produce)
FR15	Gross value shall be the graded value multiplied by the category multiplier: cereal (MZE, BNS) $\times 1.00$, perishable (POT) $\times 0.90$, cash crop (TEA) $\times 1.10$.	A tea delivery's step 3 raises the figure by a tenth; a potato delivery lowers it by a tenth.
FR16	Commission shall be 5% of the gross value.	$24,426.00 \times 0.05 = 1,221.30$.
FR17	The transport levy shall be 2.00 MUR for every kilogram delivered.	$236 \times 2.00 = 472.00$.
FR18	Net payable shall be the gross value less the commission and less the transport levy.	$24,426.00 - 1,221.30 - 472.00 = 22,732.70$.
FR19	A delivery graded REJECT shall have a gross value, a commission, a levy and a net payable of exactly zero. It shall still be recorded, and its mass shall still count towards the weekly volume figures.	A 200 kg REJECT pays 0.00, is charged 0.00, and its 200 kg appear in the weekly volume grid.
FR20	The worked example of the brief, 236 kg of BNS at a quality score of 91, shall produce an exact value of 22,732.70 MUR.	Recorded as a seeded delivery and printed at startup in objective 1.

Recording and identity

Ref	Requirement	How it is checked
FR21	Each recorded delivery shall receive a unique identifier of the form D-nnnn, issued in sequence from D-1001.	No two deliveries in a session share the same identifier
FR22	Deliveries carrying the same member identifier shall be attributed to one member, whose figures are the sum of those deliveries.	Two deliveries for M-0042 give one line in the member totals.
FR23	A recorded delivery shall not be alterable after construction.	The class exposes no setter and every field is final.

Figures on screen

Ref	Requirement	How it is checked
FR24	The program shall print the total payment per member and a season total that equals the sum of those member totals.	The printed member lines add up to the printed season total.
FR25	The program shall print a weekly volume grid of twenty weeks by four produce codes in kilograms, with row and column totals, including the mass of rejected deliveries.	A rejected 200 kg POT in week 3 appears in the POT column of row 3.
FR26	The program shall print the five highest-value deliveries, ordered by net payable descending, with ties broken by member identifier and then by week.	Two deliveries of equal value appear in member and then week order.
FR27	A search by member identifier shall return that member's figures, or a plain message when no such member exists, without crashing the application	Searching M-9999 prints a not-found message and returns to the menu.
FR28	The set of distinct member identifiers seen in the season shall determine how many member sections the report contains.	Twelve deliveries from five members produce five sections.
FR29	Rejected deliveries shall be excluded from the payment run by removing them during traversal through an Iterator, operating on a copy so that the season record and the volume figures are untouched.	After the payment run the season still holds every delivery, rejected ones included.

The season report

Ref	Requirement	How it is checked
FR30	The program shall write one Microsoft Word document at output/season-report.docx covering the whole membership.	One file appears; it opens in Word.
FR31	The report shall contain one section per member, each beginning on a new page, holding that member's delivery table, the commission, the levy, the net payable in bold, and a signature line.	Section count equals the distinct member count (FR28); each starts on a fresh page.
FR32	The report shall end with a closing section totalling the season, and those totals shall equal the sum of the member sections.	Adding the member net figures by hand gives the closing figure.
FR33	One timestamped line shall be appended to output/run-log.txt each time the report is generated.	Generating twice leaves two lines with different timestamps.
FR34	If the report cannot be written, the program shall report the cause in a	With output/ made read-only, the program names the path and the problem and keeps running.

Ref	Requirement	How it is checked
	message the user can act on and shall return to the menu.	

Non-functional requirements

Ref	Requirement	How it is checked
NFR1	Money shall be held as an unrounded double through every intermediate step and rounded to two decimals only at the moment of display.	The 88.3 kg tea delivery gives 2,476.26; rounding each step instead gives 2,476.27.
NFR2	No input the user can type shall terminate the program with a stack trace.	Text, empty lines and out-of-range numbers at all six prompts.
NFR3	The project shall build and run from a clean clone with one documented command on a JDK 21 or later.	git clone, then the single command in the README.
NFR4	Source shall be separated into the packages model, service, util and app, with no class in a package whose responsibility it does not share.	Directory listing; no console I/O in model.
NFR5	Fields shall be private, and the invariants of every domain object shall be enforced in its constructor so that an invalid object cannot be built.	Constructing a Delivery with a mass of 0 throws an error rather than returning an object.
NFR6	Lookup of a member's total or delivery list by identifier shall be constant time on average, not a scan of the season.	The lookup is a hash-based map access, not a loop.
NFR7	The polymorphic valuation path shall contain no instanceof and no downcasting.	Searching the source for either term returns nothing on that path.
NFR8	Every figure displayed or written shall be computed at run time. No figure shall be a literal.	Changing one seeded mass changes every dependent figure.
NFR9	Every refusal message shall name the acceptable range in plain English, in one line.	Read the six messages.
NFR10	The same inputs shall produce the same figures on every run: no reliance on iteration order of an unordered collection for any displayed sequence.	Two runs with the same input produce byte-identical screen output.

Noun Verb Analysis

Section one and two of the assignment brief, contains several nouns and verbs that might be crucial to building the projects. This section analyses those verbs and nouns and places them into categories. Nouns are our candidate classes while verbs are our candidate methods.

Nouns found in the brief

Cooperative, planter, member, member identifier, delivery, slip, delivery identifier, produce, maize, beans, potatoes, green tea leaf, produce code, mass, kilogram, quality score, grade, category, cereal, perishable, cash crop, base price, multiplier, value, commission, transport levy, net payable, season, week, collection point, clerk, treasurer, committee, report, payment statement, signature line, weekly volume grid, season total, exercise book, calculator

Verbs found in the brief

Weigh, score, write out (a slip), record, grade, apply (a multiplier), calculate, keep (commission), charge (levy), deduct, pay, validate, re-prompt, sort, search, remove, total, report, generate, print, sign, hand out

Candidates kept, and why

Class	Package	Why it earns a class
Delivery	model	The central noun. It has a state that varies per instance (mass, score, week, produce) and behaviour that belongs to it (its own valuation). Every other figure in the system is derived from a set of these.
Member	model	Has identity independent of any delivery instance, and is the unit the report is organised around, and owns the aggregate figures the committee asks for.
Produce (abstract Class)	model	Maize, beans, potatoes and tea share the same shape: a code, a name, a base price but differ in one behaviour, the category multiplier. That is exactly what an abstract class with one abstract method is for.
CerealProduce PerishableProduce CashCropProduce	model	The three categories of rule 3. Each supplies its own multiplier and its own display name; nothing else differs, so nothing else is overridden.
Grade (enum)	model	A closed set of four values, each carrying a number that belongs to it. An enum, not a String, because the compiler can then guarantee no fifth grade exists, and the multiplier travels with the constant instead of living in a separate lookup.
ProduceCatalog	model	The price list of rule 1, held as an array. One place where the four Produce objects are constructed with the correct subclass, and the only route from a typed code to a Produce.
Payable (interface)	model	It can state what it is worth and what is deducted from it. A delivery has it; so does a member, whose figures are the sum of theirs. The report writer needs only this view.
Reportable (interface)	model	It can describe itself as a heading and a row. Held by both the delivery, which becomes a table row, and the member, which becomes a section.
SeasonService	service	The season is not a thing with attributes; it is the work of holding and aggregating the deliveries. All of objective 3's collections live here.

Class	Package	Why it earns a class
SeasonReportWriter	service	The verbs generate, print and sign. Kept separate from the model so that no domain class knows what a .docx is.
Validator, MoneyFormat	util	The verbs validate and display. Stateless rules, gathered so that the accepted ranges and the two-decimal format each appear once.
RekoltApp, ConsoleMenu, ConsoleReader	app	The conversation with the user. Separated from the service layer so that the payment rules could be exercised by a test with no keyboard attached.

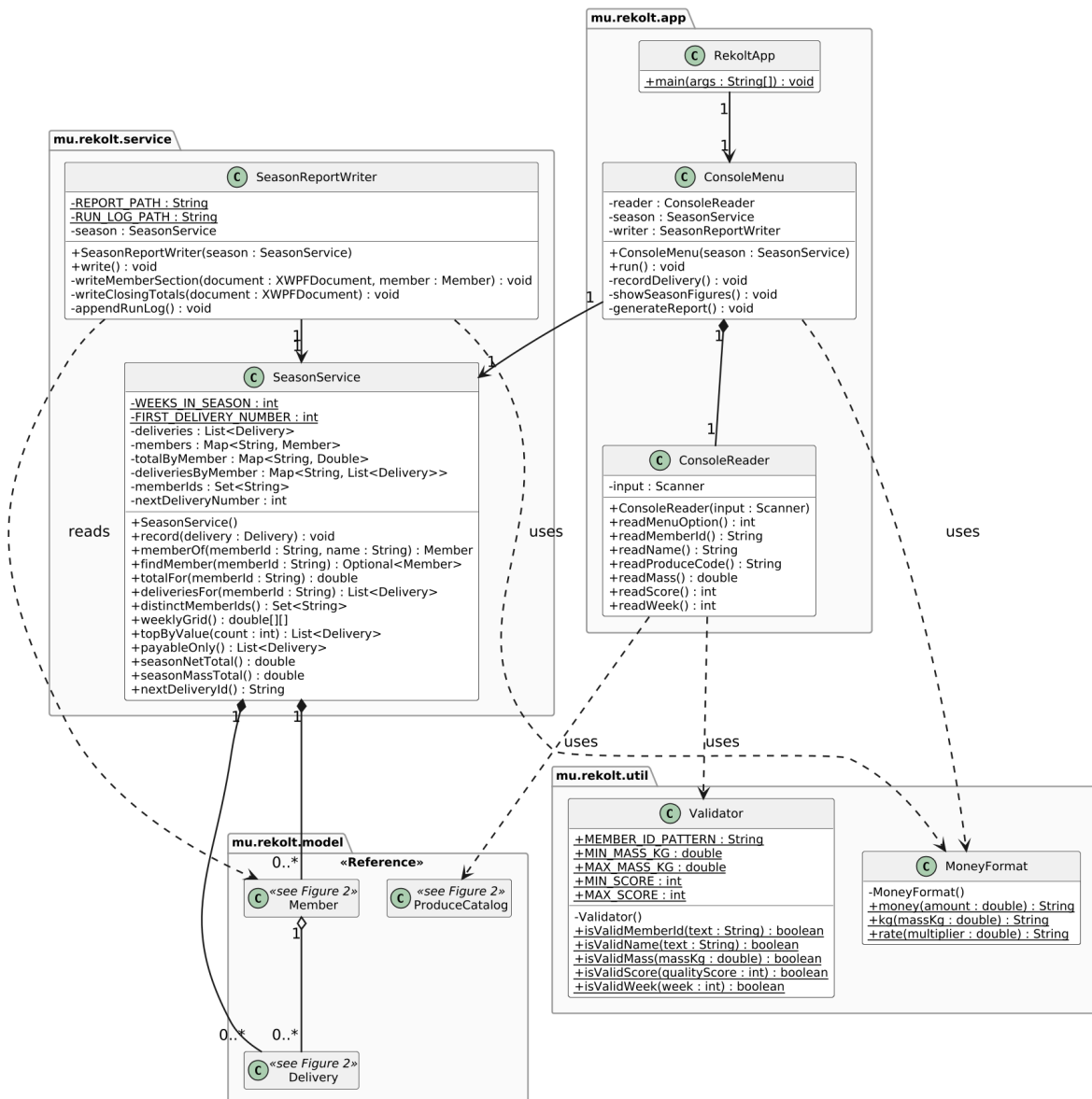
Candidates rejected, and why

Candidate noun	Reason for rejection
Clerk and Treasurer	They are the people who use the system, not data the system holds. Nothing is stored about them, no figure depends on which of them is present, and the program behaves identically to whoever is typing.
Collection point	There is one, it is fixed, and nothing in the brief varies by location. A class would hold a single constant value shared by every delivery, which is a constant, not an object.
Slip	Another word for Delivery. The slip is the paper form of the same record: identifier, member, produce, mass, score, week.
Season	Rejected as a class because it has no attributes of its own beyond the deliveries it contains, and no behaviour beyond aggregating them.
Payment statement	A rendering of a member's deliveries, not a fact the system stores. Making it a class would duplicate the figures already held on Member. It is produced on demand by SeasonReportWriter from recorded member data.
Category	Rejected because its only behaviour is the multiplier, and that is precisely what the three Produce subclasses already supply through polymorphism.
Week and Mass	Values, not objects: a whole number from 1 to 20 and a decimal number of kilograms. They have a permitted range, which the data dictionary records and the validator enforces, but no behaviour of their own. They are attributes of Delivery.

Verbs mapped to methods

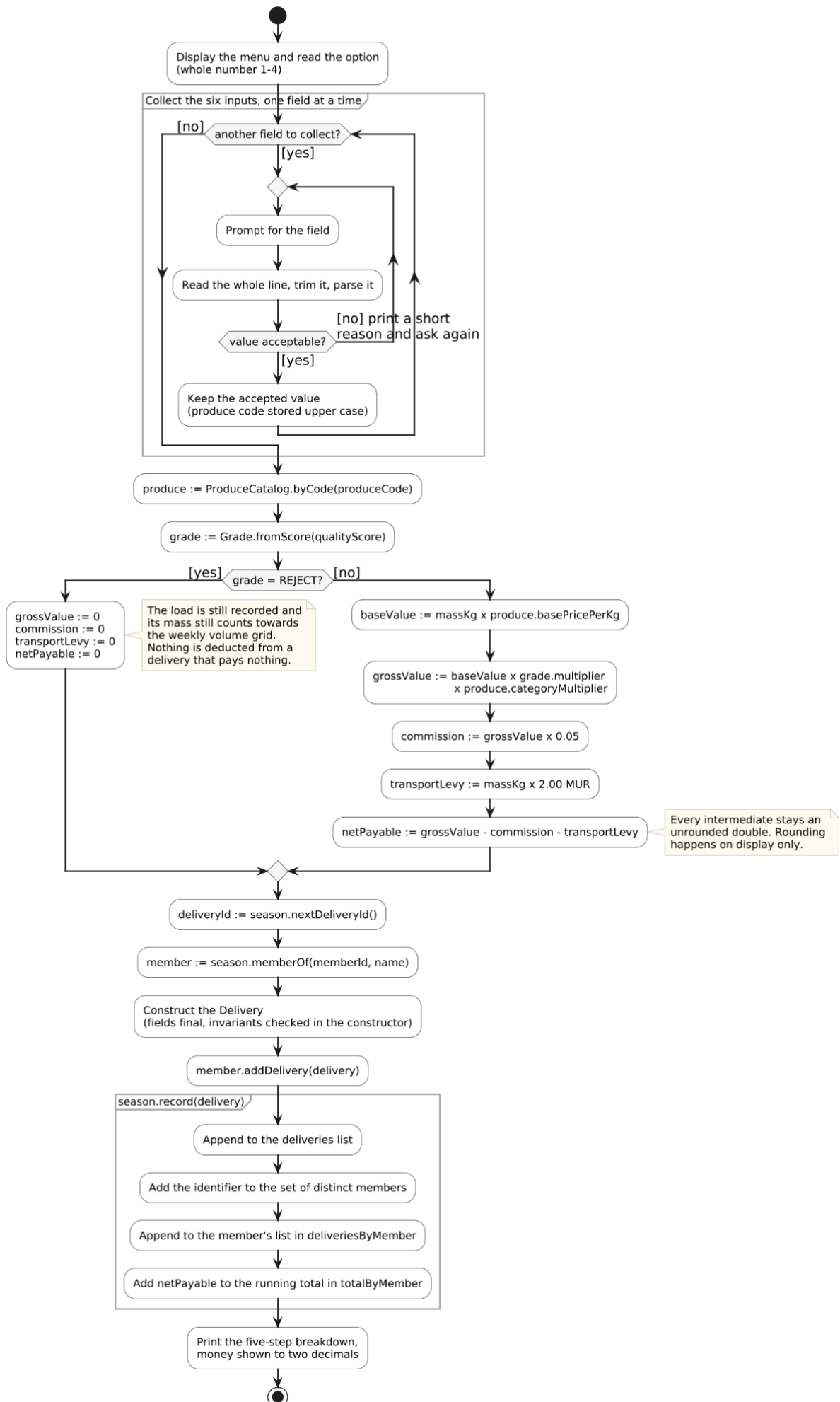
Verb	Method it became
weigh, score	ConsoleReader.readMass(), ConsoleReader.readScore() — the clerk's two acts enter the system as validated input.
grade	Grade.fromScore(int) : Grade
apply a multiplier	Grade.getMultiplier(), Produce.categoryMultiplier()
calculate, deduct, keep, charge	Delivery.grossValue(), commission(), transportLevy(), netPayable()
record	SeasonService.record(Delivery)
total	Member.netPayable(), SeasonService.seasonNetTotal()

UML Class Diagram Layers



Behavioural diagram: recording a delivery

Recording one delivery (menu option 1)



Pseudocode: The net payment

This pseudocode showcases how the net payment for a member farmer is computed from the moment their produce is submitted. The REJECT case is handled by an early exit before any deduction is computed, which guarantees that a rejected load is neither paid nor charged.

Constant

```
CommissionRate = 0.05      // rule 4
LevyPerKg      = 2.00 MUR  // rule 5
```

Function gradeOf(qualityScore)

```
If qualityScore >= 85 Then Return A      // 85-100, x1.15
Else If qualityScore >= 70 Then Return B  // 70-84, x1.00
Else If qualityScore >= 50 Then Return C  // 50-69, x0.85
Else                                     // 0-49, x0.00
    Return Reject
END
```

Function netPayable(massKg, qualityScore, produceCode)

```
// Key assumption: massKg, qualityScore and produceCode have already
// passed validation. this function never re-checks them.
```

```
produce <- ProduceCatalog.byCode(produceCode)
grade  <- gradeFor(qualityScore)
```

```
// the REJECT branch
```

```
IF grade = REJECT THEN
    baseValue <- massKg x produce.basePricePerKg
    gradedValue <- 0.00
    grossValue <- 0.00
    commission <- 0.00
    transportLevy <- 0.00
    netPayable <- 0.00
    RETURN netPayable = 0.00 MUR
```

```
// the paying branch
```

```
baseValue <- massKg x produce.basePricePerKg      // step 1
gradedValue <- baseValue x grade.multiplier         // step 2
grossValue <- gradedValue x produce.categoryMultiplier // step 3
commission <- grossValue x CommissionRate          // step 4
transportLevy <- massKg x LevyPerKg                // step 5
netPayable <- grossValue - commission - transportLevy
```

```
RETURN netPayable
END FUNCTION
```

Data dictionary

Values entered by the user

Data item	Type	Unit	Permitted range	Validation rule
menuOption	int	—	1, 2, 3 or 4	Whole line parsed with Integer.parseInt; a parse failure or a value outside the range prints the range and re-prompts.
memberId	String	—	Matches M-\d{4}, e.g. M-0042	Whole-string regular expression matches, so M-00423 fails rather than matching in part.
memberName	String	—	At least one non-whitespace character	Trimmed, then tested non-empty, which rejects a whitespace-only line for free. Validator.isValidName()
produceCode	String	—	MZE, BNS, POT, TEA	Trimmed and upper-cased at the prompt, then matched against the catalog; the upper-case form is what is stored.
massKg	double	kilogram	> 0.0 and ≤ 5000.0 (exclusive lower, inclusive upper)	double because kilograms are measured, not counted, and a load can weigh 236.5 kg. Parsed with Double.parseDouble inside a try/catch.
qualityScore	int	—	0 to 100 inclusive	int because the clerk awards whole marks. Parsed with Integer.parseInt, which rejects “72.5” outright rather than truncating it.
week	int	week number	1 to 20 inclusive	Whole number, bounded by the twenty-week season mentioned in the brief

Values the program derives

Data item	Type	Unit	Range	Rule
deliveryId	String	—	D-1001 upwards, unique within a run	Issued in sequence by SeasonService.nextDeliveryId(); never reused, never re-issued after a deletion because there are no deletions.
grade	Grade (enum)	—	A, B, C, REJECT	Derived once from qualityScore in the constructor and stored, so a delivery cannot grade two ways.
gradeMultiplier	double	factor	1.15, 1.00, 0.85, 0.00	A field of the enum constant, not a lookup table. Read-only.
basePricePerKg	double	MUR /kg	30.00, 90.00, 45.00, 25.00	Set once when ProduceCatalog builds the four objects; final thereafter.
categoryMultiplier	double	factor	1.00, 0.90, 1.10	Returned by the Produce subclass; not stored as data anywhere.
baseValue	double	MUR	0.00 to 450,000.00 (5000 kg × 90)	massKg × basePricePerKg. Unrounded.

Data item	Type	Unit	Range	Rule
gradedValue	double	MUR	0.00 to 517,500.00	$\text{baseValue} \times \text{gradeMultiplier}$. Unrounded.
grossValue	double	MUR	0.00 to 517,500.00	$\text{gradedValue} \times \text{categoryMultiplier}$. Zero for a REJECT. Unrounded.
commission	double	MUR	0.00 to 25,875.00 (5% of the largest possible gross)	Zero when the grade is REJECT, otherwise $\text{grossValue} \times 0.05$.
transportLevy	double	MUR	0.00 to 10,000.00 (5000 kg $\times$ 2)	Zero when the grade is REJECT, otherwise $\text{massKg} \times 2.00$. This suppression is explicit; it does not fall out of the arithmetic.
netPayable	double	MUR	0.00 upwards; never negative	$\text{grossValue} - \text{commission} - \text{transportLevy}$. Cannot go negative: for any paying grade the levy is at most 2 MUR/kg against a base price of at least 25 MUR/kg
displayed money	String	MUR	two decimals, thousands separated	MoneyFormat.money(), String.format("%.2f", ...). The only place any rounding happens.

Values the program holds

Structure	Type	Size	Contents and purpose
deliveries	ArrayList<Delivery>	0 to like 10-15	Every delivery of the season in the order recorded, rejected ones included. The single source from which every other figure is derived.
members	HashMap<String, Member>	0 to like 10-15 entries	Member identifier to the one Member object for it, so the same planter is never held twice.
totalByMember	HashMap<String, Double>	0 to like 10-15 entries	Running net payable per member, updated on each record so the report needs no re-computation.
deliveriesByMember	HashMap<String, List<Delivery>>	0 to like 10-15 keys	The deliveries of each member, which become the rows of that member's table in the report.
memberIds	HashSet<String>	0 to like 10-15 entries	The distinct identifiers seen; its size is the number of sections the report must contain.
Catalog	Produce[]	exactly 4	The price list of rule 1. An array because the length is fixed and known at compile time.
weeklyGrid	double[][] (2d array)	20 $\times$ 4	Kilograms per week per produce code. Rectangular and fixed, which is what a two-dimensional array is for. Week n is stored in row $n-1$.
ProduceCode	String[]	exactly 4	Column order of the grid; fixes the order of the headings everywhere.

Output paths

Item	Type	Value and rule
ReportPath	String, static final	output/season-report.docx. Overwritten on each generation.
RunLogPath	String, static final	output/run-log.txt. Appended to, never overwritten; one ISO-8601 timestamped line per generation.

Design Rationale

Total payment per member: `HashMap<String, Double>`

Access pattern: it is written once per delivery while doing additions for totals and then accessed by member identifier when searching for a specific member or generating a season report(upcoming objective)

Ordering: None is needed, and a HashMap does not provide a specific order. Where an order is wanted on screen the identifiers are copied into a list and sorted by a comparator.

Lookup cost: searching for a member takes $O(1)$ time on average since it searches for a unique identifier per member. Thus, even when the number of members increase, the number of operations done stays constant.

Rejected alternative: `TreeMap<String, Double>`. It keeps its keys sorted, which costs more time per lookup instead of constant. Sorted by identifier is not an order this application needs because the totals table is sorted by amount and the search is by exact key.

Distinct member identifiers: `HashSet<String>`

Access pattern: it stores recorded member ids and compares them to identify if each is unique to a member and how many they are.

Ordering: there is no particular ordering rule followed as long as each id stored is unique. Any sorting is done on a new copy ArrayList of the original HashSet.

Lookup cost: it takes $O(1)$ time to add or search for a member id because each one of them is unique. There is no moving through the whole list

Rejected alternative: a `List<String>` which contains a check before each add. The problem is that searching the whole list would not be efficient because it would have to look for distinct members list and the deliveries list which would take $O(n^2)$ time.